

Programmation Orientée Objet pour le Calcul Scientifique : Part I Survival kit

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- Outils et Langage (Tool)
- **Orienté objet (OO)**
- Gestion de la mémoire (Mem)
- Programmation statique (Static)

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 6 : Une première forme de polymorphisme

Les fonctions peuvent être **surchargées** 1/2

Elles pourront changer de comportement en fonction de leurs arguments :

```
void hello() {std::cout << "Hello world";}  
void hello(int year) { std::cout <<"Hello year " << year;}
```

Attention :

- le type de retour n'est pas discriminant

```
void hello() {...}  
int hello() // ERROR : function hello already declared
```

Item 6 : Une première forme de polymorphisme

Les fonctions peuvent être **surchargées** 2/2

Elles pourront changer de comportement en fonction de leurs arguments :

```
void hello() {std::cout << "Hello world";}  
void hello(int year) { std::cout <<"Hello year " << year;}
```

Exo 6.1 Créer une fonction `multiple` qui multiplie deux entiers l'un par l'autre.

Surchargez cette fonction pour qu'elle fonctionne avec un seul entier en entrée (elle le multipliera alors par 10).

Item 7 : créer une classe/un objet en C++

Qu'est-ce qu'une **classe** :

- une structure qui contient des données et des traitements sur ces données

```
class MyClass {};
```

Qu'est-ce qu'un **objet** :

- une **instance** de classe, ie. une variable de type `MyClass`

```
MyClass my_object;
```

Item 7 : créer une classe/un objet en C++

Comment s'écrit une **classe** :

```
class <MyClassName> {  
    <data>;  
    <methods>;  
};
```

```
class MyClass {  
    int i =0;  
    void print() {std::cout << i;}  
};
```

Le mot clé **struct** peut également s'utiliser. Voir plus loin (Visibilité).

Item 7 : créer une classe/un objet en C++

Comment s'utilise une **classe**

- on accède aux données et aux méthodes par l'opérateur `.`

```
MyClass my_object{};  
my_object.i;  
my_object.print();
```

Exo 7.1 Créer votre propre classe avec une donnée et une méthode (mot clé `class`). Créer un objet de cette classe. Essayer d'accéder aux champs via l'opérateur `.` : que constatez-vous ?

Item 7 : créer une classe/un objet en C++

Notion de visibilité 1/3

Comme l'Exo 7.1 vous le montre :

- les champs (données ou méthodes) d'une classe sont, par défaut, **inaccessibles**

Avant de déclarer un bloc dans une classe il faut indiquer sa **visibilité**

```
class <MyClassName> {  
    <visibility>:  
    <data>;  
    <visibility>:  
    <methods>;  
};
```

<visibility> = private / public ou protected (vu plus tard)

Item 7 : créer une classe/un objet en C++

Notion de visibilité 2/3

- `private:` signifie que les champs ne seront accessibles qu'à l'intérieur de la classe
 - ie. dans les autres méthodes de la classe.
- `protected` identique à `private` + accessibles à l'intérieur d'une classe dérivée
- `public:` signifie que les champs seront accessibles à l'extérieur de la classe

On observe en pratique que :

- les **données** sont le plus souvent privées
- les **méthodes** sont le plus souvent publiques

Cela permet de proposer des **traitements sans exposer la représentation interne des données.**

Item 7 : créer une classe/un objet en C++

Notion de visibilité 3/3

```
class MyClass {  
private:  
    int i = 0;  
public:  
    void print() {std::cout << i;}  
};  
int main(){  
    MyClass c{};  
    c.print(); // OK  
    c.i = 2; // ERROR  
}
```

Exo 7.2 Modifier votre classe de l'Exo 7.1 pour donner accès à votre méthode, sans exposer vos données.

Item 7 : créer une classe/un objet en C++

Visibilité par défaut deux mots clés `class` & `struct` pour construire une classe

- `class` : visibilité `private` par défaut ;
- `struct` : visibilité `public` par défaut.

```
class MyClass {  
    int i = 0; // private  
};  
struct MyStruct {  
    int i = 0; //public  
};
```

Item 7 : créer une classe/un objet en C++

Changer la **Visibilité** par amitié avec le mot clé `friend` 1/2

```
class MyClass {  
    int i = 0; // private  
    friend void my_friend_function(MyClass mc) {  
        std::cout << mc.i << std::endl;  
    }  
};  
  
int main() {  
    MyClass mc {};  
    my_friend_function(mc); // print "0"  
    std::cout << mc.i << std::endl; // ERROR 'int MyClass::i' is private within this context  
}
```

Item 7 : créer une classe/un objet en C++

Changer la **Visibilité** par amitié avec le mot clé `friend` 2/2

```
class MyClass {
    int i = 0; // private
    friend class MyFriendClass;
};
class MyFriendClass {
public:
    void printMyClass(MyClass mc) {
        std::cout << mc.i << std::endl;
    }
};

int main() {
    MyClass mc {};
    MyFriendClass mfc{}; // print "0"
    mfc.printMyClass(mc);
    std::cout << mc.i << std::endl; // ERROR 'int MyClass::i' is private within this context
}
```

Item 7 : créer une classe/un objet en C++

Membres ou méthodes non mutables : `const` 1/3

- un membre `const` reste constant pendant toute la durée de vie de l'objet
 - il doit être initialisé !

```
struct MyStruct {  
    int const i = 10;  
};  
  
int main(){  
    MyStruct a;  
    a.i = 0; //ERROR  
}
```

Item 7 : créer une classe/un objet en C++

Membres ou méthodes non mutables : `const` 2/3

- une méthode `const` ne peut pas modifier des membres de l'objet

```
struct MyClass {  
    int m_i = 10;  
    void print() const;  
};  
  
void MyClass::print() const {  
    m_i = 10; // ERROR  
}
```

Exo 7.3 Ajouter à votre classe une méthode `const` manipulant une donnée `const`.

Item 7 : créer une classe/un objet en C++

Membres ou méthodes non mutables : `const` 3/3

- Un objet peut être déclaré `const` :
 - on ne peut alors appeler que ses méthodes `const`

```
class MyClass {  
private:  
    int m_i = 0;  
public:  
    void non_const_method() {}  
    void const_method() const {}  
};  
int main(){  
    MyClass const a{};  
    a.const_method(); // OK  
    a.non_const_method(); // ERROR  
}
```

Item 7 : créer une classe/un objet en C++

Pointeur d'instance `this`

- Il est possible d'accéder à l'objet au sein des méthodes via le pointeur `this`

```
class MyClass {  
private:  
int m_i = 0;  
public:  
void increment() {  
    m_i++;  
    // equivalent to  
    this->m_i++; // Note operator -> equivalent to . for pointers  
}  
};
```

Item 7 : créer une classe/un objet en C++

Classe et fichiers 1/6

Une bonne architecture de classe se compose :

- d'un **fichier header** (.h) contenant la **déclaration** de la classe, données/méthodes
- d'un **fichier source** (.cpp) contenant les **définitions** des méthodes

Item 7 : créer une classe/un objet en C++

Classe et fichiers 2/6

Une bonne architecture de classe se compose :

- d'un **fichier header** (.h) contenant la **déclaration** de la classe, données/méthodes

```
// MyClass.h
class MyClass {
    int m_i = 0;
public:
    void print() const;
};
```

Item 7 : créer une classe/un objet en C++

Classe et fichiers 3/6

Une bonne architecture de classe se compose :

- d'un **fichier source** (.cpp) contenant les **définitions** des méthodes

```
// MyClass.cpp
#include "MyClass.h"
void MyClass::print() const {
    std::cout << "i = " << m_i;
}
```

- **Exo 7.4** : créer une classe décrite dans deux fichiers .h et .cpp .

Item 7 : créer une classe/un objet en C++

Classe et fichiers : Include guard 4/6

- Un fichier `.h` ne peut pas être inclus deux fois dans le même fichier

```
// MyOtherClass.h
#include "MyClass.h"

// main.cpp
#include "MyClass.h"
#include "MyOtherClass.h" //ERROR class MyClass already defined
```

- on protège donc un fichier `.h` par des directives de compilation
`#ifndef`, `#endif`, `#define` pour ne l'inclure qu'une seule fois

Item 7 : créer une classe/un objet en C++

Classe et fichiers : Include guard 5/6

- Exemple de protection du fichier `MyClass.h`

```
// MyClass.h
#ifndef MYCLASS_H_
#define MYCLASS_H_
class MyClass {
    int m_i = 0;
public:
    void print() const;
};
#endif // MYCLASS_H_

// main.cpp
#include "MyClass.h"
#include "MyOtherClass.h" // OK MyClass.h not included again
```

Item 7 : créer une classe/un objet en C++

Classe et fichiers : Include guard 6/6

- **Exo 7.5** Créer un nouvel header `Tools.h`, déclarant par exemple un paramètre ou une classe, que vous inclurez dans `Myclass.h` et dans `main.cpp`.
- Remarque : en pratique vous n'écrirez pas ces protections à la main, tous les IDE modernes l'écrivent automatiquement à la création d'un fichier `.h`.

Item 8 : une affaire d'héritage

Les classes peuvent hériter les unes des autres.

- L'héritage s'écrit avec l'opérateur ":" `class Son : Parent {};`
- L'héritage donne accès aux méthodes et aux données du(des) parent(s)

```
struct Parent{
    int parent_data = 1;
    void print_parent(){std::cout << "Parent class";}
};
struct Son : Parent {};

int main(){
    Son s{};
    std::cout << s.parent_data << std::endl;
    s.print_parent();
}
```

Item 8 : une affaire d'héritage

L'héritage donne accès méthodes et données du(des) parent(s)...ou pas

- Cela dépend de la visibilité dans la classe parent : ex. `private`

```
struct Parent{
private:
    int parent_data = 1;
    void print_parent(){std::cout << "Parent class";}
};

struct Son : Parent {};

int main(){
    Son s{};
    std::cout << s.parent_data << std::endl; // KO 'Parent::parent_data' is private
    s.print_parent(); // KO 'Parent::print_parent()' is private
}
```

Item 8 : une affaire d'héritage

L'héritage donne accès méthodes et données du(des) parent(s)...ou pas

- Cela dépend de la visibilité dans la classe parent : ex. `protected`

```
class Parent{
protected :
    int parent_data = 1;
    void print_parent(){std::cout << "Parent class";}
};
struct Son : Parent {
    void print() {print_parent(); std::cout << parent_data;} // OK
};
int main(){
    Son s{};
    std::cout << s.parent_data << std::endl; // KO : protected not visible outside
    s.print_parent(); // KO : idem
    s.print(); // OK protected data called inside child Son::print
}
```

Item 8 : une affaire d'héritage

L'héritage donne accès méthodes et données du(des) parent(s)...ou pas

- Cela dépend du type d'héritage
- Celui-ci peut-être :
 - `public` (le défaut pour une `struct`)
 - `private` (le défaut pour une `class`)
 - `protected`

Exercice : refaire les trois exemples ci-dessus en changeant le type d'héritage (`public` , `protected` ou `private`).

Item 8 : une affaire d'héritage

L'héritage `public` ne modifie pas la visibilité des membres du parent chez l'enfant

```
struct Parent{
private :
    int internal_data = 0;
protected :
    int parent_data = 1;
public :
    void print_parent(){std::cout << "Parent class";}
};
class Son : public Parent { // ***** Son inherits the members with the following visibility *****
// This is pseudo-code showing how Son access its Parent data and methods
private :
    int internal_data = 0;
protected :
    int parent_data = 1;
public :
    void print_parent(){std::cout << "Parent class";}
    // *****
};
```

Item 8 : une affaire d'héritage

L'héritage `protected` rend les membres `public` du parent `protected` chez l'enfant

```
struct Parent{
private :
    int internal_data = 0;
protected :
    int parent_data = 1;
public :
    void print_parent(){std::cout << "Parent class";}
};
class Son : protected Parent {// ***** Son inherits the members with the following visibility *****
// This is pseudo-code showing how Son access its Parent data and methods
private :
    int internal_data = 0;
protected :
    int parent_data = 1;
protected :
    void print_parent(){std::cout << "Parent class";}
    // *****
};
```

Item 8 : une affaire d'héritage

L'héritage `private` rend tous les membres du parent `private` chez l'enfant

```
struct Parent{
private :
    int internal_data = 0;
protected :
    int parent_data = 1;
public :
    void print_parent(){std::cout << "Parent class";}
};
class Son : private Parent {// ***** Son inherits the members with the following visibility *****
// This is pseudo-code showing how Son access its Parent data and methods
private :
    int internal_data = 0;
private :
    int parent_data = 1;
private :
    void print_parent(){std::cout << "Parent class";}
// *****
};
```

Item 8 : une affaire d'héritage

Membres classe parent	Type d'héritage	Membres classe enfant
<code>private</code> <code>protected</code> <code>public</code>	<code>: private</code>	<code>private</code> <code>private</code> <code>private</code>
<code>private</code> <code>protected</code> <code>public</code>	<code>: protected</code>	<code>private</code> <code>protected</code> <code>protected</code>
<code>private</code> <code>protected</code> <code>public</code>	<code>: public</code>	<code>private</code> <code>protected</code> <code>public</code>

Item 8 : une affaire d'héritage

L'héritage donne accès aux méthodes et aux données du(des) parent(s)...ou pas

- **Exo 8.1** Créer les classe `Parent` et `Son` pour faire fonctionner le code ci-dessous

```
int main(){
    Parent p{42};
    p.print_parent(); // output : "Parent ; v = 42"
    std::cout << "parent value " << p.v << std::endl; // output : "parent value 42"
    Son s{};
    s.set_value(42);
    s.print(); // output "The Child, Parent ; v =42"
    s.print_parent(); // output : "Parent ; v = 42"
    std::cout << s.v; // output : "42"
}
```

Modifier `Son` pour que les deux dernières lignes ne fonctionnent pas, puis pour que les "enfants" de `Son` n'accèdent pas aux données/méthodes de `Parent`.

Item 8 : une affaire d'héritage

L'héritage permet de se faire passer pour...

```
struct Base{
    void print() {std::cout << "Base"<< std::endl;}
};
struct Derived : public Base{
    void print() {std::cout << "Derived"<< std::endl;}
};

int main(){
    Derived d{};
    d.print(); // "Derived"
    Base& b = d;
    b.print(); // "Base"
}
```

C'est ce que l'on appelle le **polymorphisme**, l'objet est à la fois **Base** et **Derived**

Item 8 : une affaire d'héritage

L'héritage permet de redéfinir (surcharger) le comportement du parent (polymorphe)

```
struct Base{
    virtual void print() {std::cout << "Base"<< std::endl;}
};
struct Derived : public Base{
    void print() override {std::cout << "Derived"<< std::endl;}
};

int main(){
    Derived d{};
    d.print(); // "Derived"
    Base& b = d;
    b.print(); // "Derived"
}
```

Attention dans ce cas le destructeur de la classe de base doit être **virtuel**.

Item 8 : une affaire d'héritage

L'héritage permet de redéfinir (surcharger) le comportement du parent

Destructeur de classe de base virtuel ou risque de non destruction de la classe dérivée

```
struct Base{
    virtual void print() {std::cout << "Base"<< std::endl;}
    virtual ~Base() = default;
};
struct Derived : public Base{
    void print() override {std::cout << "Derived"<< std::endl;}
};

int main(){
    Base* b = new Derived{};
    delete b; // delete Derived only if ~Base is virtual...
}
```

Item 8 : une affaire d'héritage

L'héritage permet de redéfinir (surcharger) le comportement du parent

Attention ce n'est possible que pour un héritage public

```
struct Base{
    virtual void print() {std::cout << "Base"<< std::endl;}
    virtual ~Base() = default;
};
struct Derived : private Base{
    void print() override {std::cout << "Derived"<< std::endl;}
};

int main(){
    Derived d{};
    d.print(); // "Derived"
    Base& b = d; // KO : Base is an inaccessible base of Derived
}
```

Item 8 : une affaire d'héritage

- L'héritage donne accès aux méthodes et aux données du(des) parent(s)
 - à l'intérieur de la classe dérivée (tous les types d'héritages)
 - à l'extérieur de la classe dérivée (héritage `public`)
- L'héritage, s'il est `public` , permet de :
 - se faire passer pour...
 - redéfinir (surcharger) le comportement du parent

Exo 8.2 Construire une classe de base `Point` et deux classes dérivées `Point2D` et `Point3D` (ne contenant qu'une méthode `print` affichant le nom de la classe). Afficher les noms des classes dérivées `Point2D` et `Point3D` mais depuis une fonction manipulant un `Point&` .